HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO TUẦN 2

Đề tài: Bài toán nhận diện biển số xe

Giảng viên hướng dẫn:	Kim Ngọc Bách
Nhóm lớp	11
Họ và tên sinh viên	Đinh Hoàng Vũ
Mã sinh viên	B23DCCN941

Hà Nội – 2026

Mục lục

I. Object Detection 3
1. YOLO v8 3
1.2. Cách hoạt động của YOLOv8 3
1.3. Ví dụ sử dụng YOLOv8 5
II. Thư viện OpenCV 6
III. Image Preprocessing 6
1. Các biến đổi hình học 6
1.1. Phóng đại ảnh (Scale ảnh) 7
1.2. Dịch chuyển ảnh 7
1.3. Xoay ảnh (Rotation) 7
1.4. Biến đổi Affine 8
1.5. Biến đổi phối cảnh 8
2. Làm mịn ảnh 8
2.1. Khái niệm 8
2.2. Nguyên lý hoạt động 8
2.3. Các phương pháp làm mịn phổ biến 9
2.4. Vai trò của làm mịn ảnh trong xử lý ảnh 9

BÁO CÁO TUẦN HỌC PHẦN THỰC TẬP CƠ SỞ

MỤC TIÊU: Tiếp tục tìm hiểu các lý thuyết liên quan để phục vụ cho bài toán

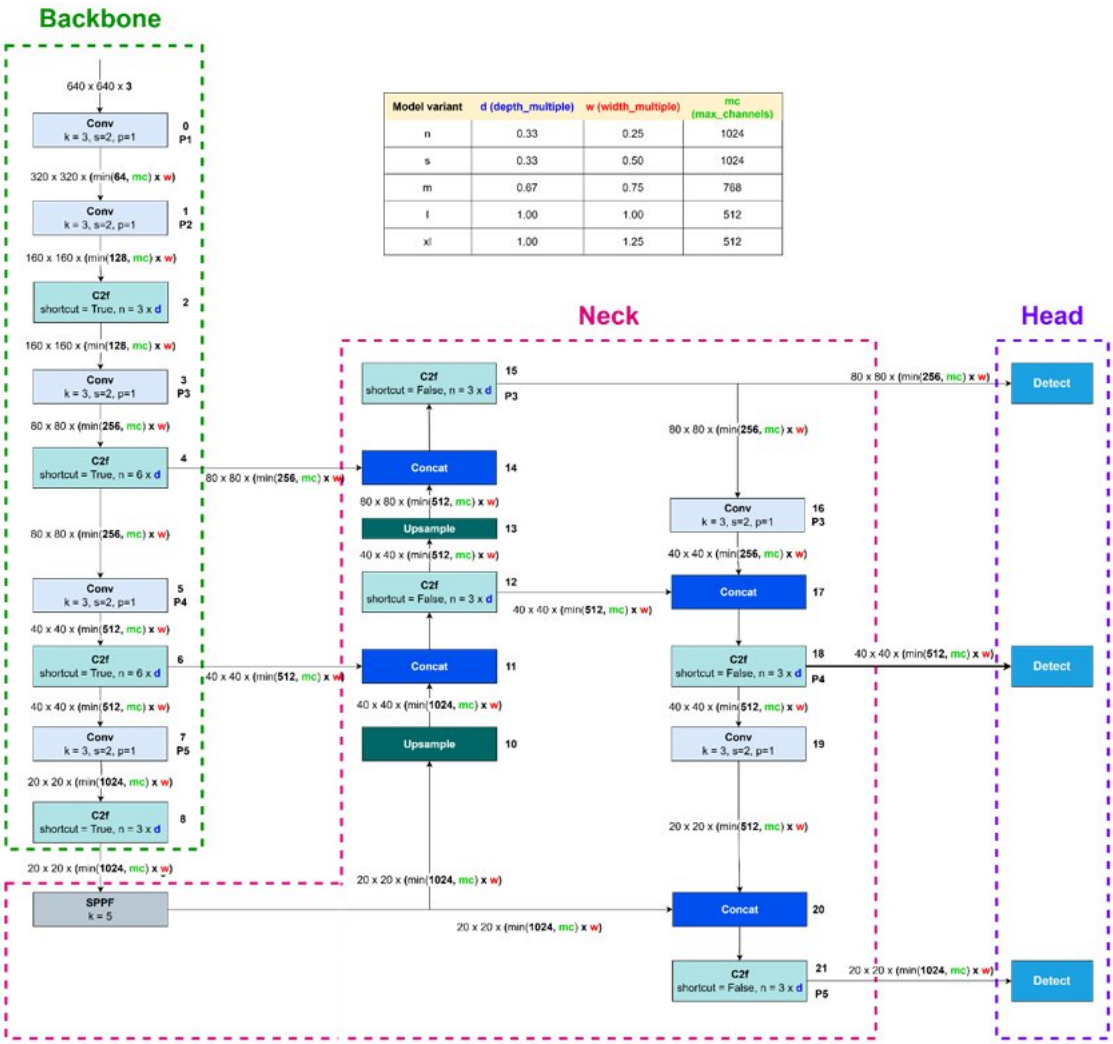
I. Object Detection

1. YOLO v8

YOLO (You Only Look Once) là một thuật toán phát hiện đối tượng trong hình ảnh và video được phát triển bởi Joseph Redmon, Ali Farhadi và Oluwatimilewe vào năm 2016. Nó nhanh chóng trở thành một trong những mô hình Object Detection phổ biến nhất hiện nay nhờ vào tốc độ xử lý nhanh và độ chính xác cao

YOLOv8 là phiên bản mới nhất của YOLO, được biết đến với khả năng phát hiện đối tượng trong thời gian thực.

1.2. Cách hoạt động của YOLOv8



Model variant	d (depth_multiple)	w (width_multiple)	mc (max_channels)
n	0.33	0.25	1024
s	0.33	0.50	1024
m	0.67	0.75	768
l	1.00	1.00	512
xl	1.00	1.25	512

1.2.1. Backbone (Trích xuất đặt trưng)

Backbone là thành phần đầu tiên của mô hình, có nhiệm vụ trích xuất đặt trưng (feature extraction) từ ảnh đầu vào. Ảnh sau khi được đưa vào mạng sẽ đi qua

nhiều lớp tích chập (**Convolutional Layers**) để tạo ra các **feature map** chứa thông tin quan trọng về hình dạng, kết cấu và vị trí của đối tượng.

YOLOv8 sử dụng các lớp:

- Convolution
- Batch Normalization
- Hàm kích hoạt SiLU (Sigmoid Linear Unit)

Một điểm cải tiến quan trọng của **YOLOv8** là sử dụng **C2f block** thay cho các khối C3 được sử dụng trong các phiên bản trước. **C2f block** giúp cải thiện khả năng lan truyền gradient, giảm số lượng tham số và tăng hiệu suất tính toán của mạng.

Cấu trúc cơ bản của C2f block bao gồm:

- Chia feature map đầu vào thành nhiều nhánh
- Áp dụng các lớp bottleneck trên từng nhành
- Kết hợp các đặc trưng lại với nhau

Ngoài ra, phần cuối của backbone sử dụng **Spatial Pyramid Pooling Fast (SPPF)** để mở rộng vùng tiếp nhận của mạng, giúp mô hình học được thông tin ngữ cảnh ở nhiều mức khác nhau

1.2.2. Neck (Kết hợp đặc trưng)

Neck đóng vai trò tổng hợp và kết hợp các đặc trưng từ nhiều tầng khác nhau của backbone nhằm phát hiện đối tượng ở các kích thước khác nhau.

YOLOv8 sử dụng kết hợp hai cấu trúc phổ biến:

Feature Pyramid Network (FPN)

FPN truyền thông tin từ tầng sâu của mạng xuống các tầng nông hơn thông qua quá trình **upsamling**, giúp các feature có thêm ngữ cảnh.

Path Aggregation Network (PAN)

PAN giúp truyền thông tin theo chiều ngược lại, từ các tầng nông lên các tầng sâu thông qua quá trình **downsampling**. Điều này giúp cải thiện khả năng kết hợp thông tin giữa các mức đặt trưng khác nhau.

Sau khi đi qua neck, mô hình tạo ra nhiều feature map ở các độ phân giải khác nhau, thường bao gồm:

- Feature map độ phân giải các để phát hiện đối tượng nhỏ
- Feature map trung bình cho các đối tượng kích thước trung bình
- Feature map độ phân giải thấp cho đối tượng lớn

Cơ chế này được gọi **là multi-scale detection.**

1.2.3. Detection Head (Phát hiện đối tượng)

Dectection head là phần cuối của mô hình, chịu trách nhiệm dự đoán các thông tin về đối tượng.

YOLOv8 sử dụng Decoupled Head, nghĩa là tách riêng hai nhiệm vụ:

- **Classification branch** – dự đoán đối tượng
- **Bounding box regression branch** – dự đoán vị trí bounding box

Việc tách hai nhiệm vụ này giúp giảm xung đột giữa các mục tiêu học khác nhau và cải thiện đội chính xác của mô hình

1.2.4. Anchor-Free Prediction

Một cải tiến quan trọng của YOLOv8 là sử dụng **anchor-free detection** thay vì phương pháp **anchor-based** được sử dụng trong các phiên bản YOLO trước.

Trong phương pháp anchor-based, mô hình phải sử dụng các anchor box cố định để dự đoán vị trí đối tượng. Tuy nhiên, việc lựa chọn kích thước anchor phù hợp thường phức tạp và cần nhiều quá trình tinh chỉnh.

Trong YOLOv8, mỗi điểm trên feature map sẽ dự đoán trực tiếp khoảng cách đến bốn cạnh của bounding box, bao gồm: **left, right, top, bottom**

Phương pháp này giúp giảm số lượng siêu tham số và làm cho quá trình huấn luyện trở nên đơn giản hơn.

1.2.5. Lọc kết quả

Sau khi dự đoán, mô hình có thể tạo ra nhiều bounding box chồng lên nhau.

Thuật toán **Non-Maxinum Suppression (NMS)** sẽ được sử dụng để chọn ra bounding bõ có **confidence** cao nhất và loại bỏ các các box trùng lặp có IoU lớn

1.3. Ví dụ sử dụng YOLOv8

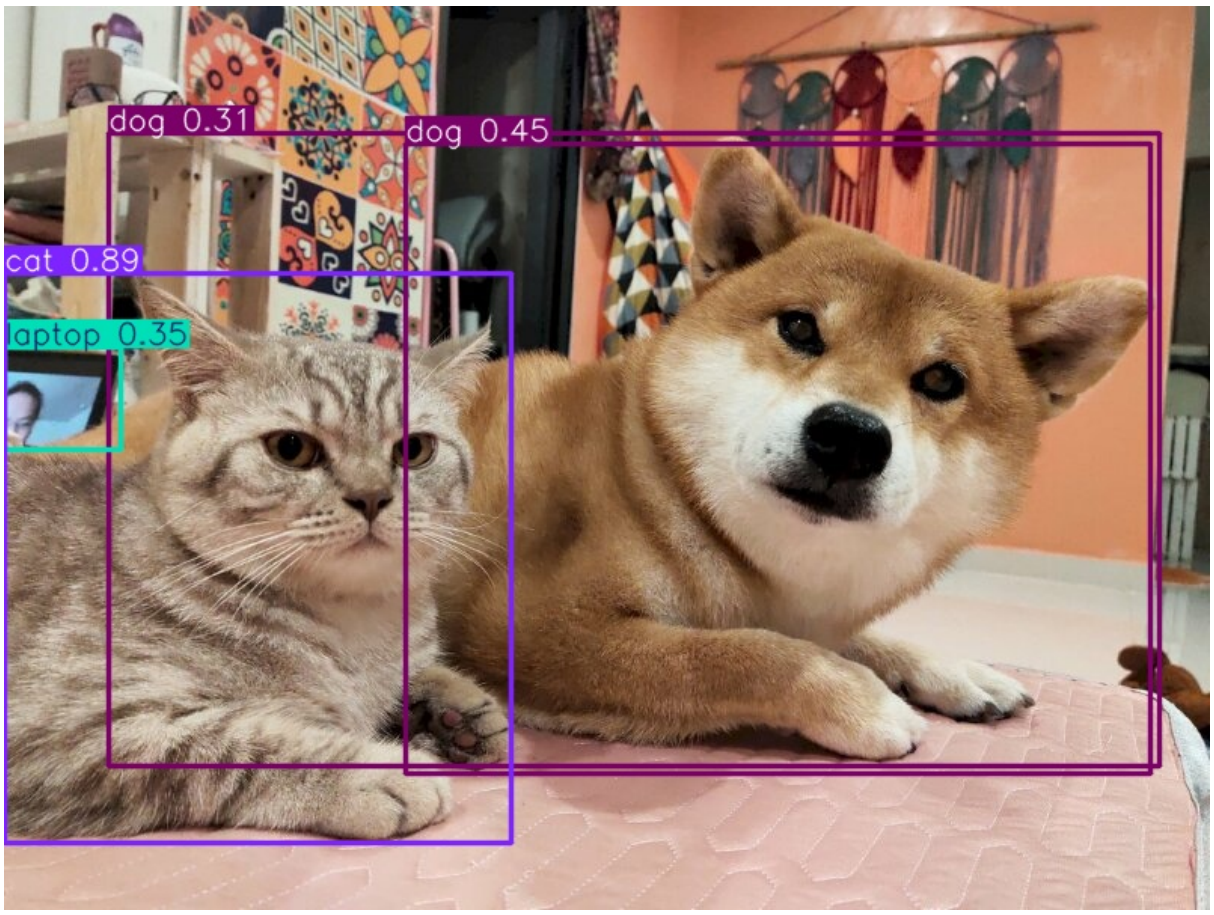
```
from ultralytics import YOLO

# Load a COCO-pretrained YOLOv8n model
model = YOLO("yolov8n.pt")

# Display model information (optional)
model.info()

# Train the model on the COCO8 example dataset for 100 epochs
results = model.train(data="coco8.yaml", epochs=100, imgsz=640)

# Run inference with the YOLOv8n model on the 'bus.jpg' image
results = model("dogcat.png", save=True)
```



II. Thư viện OpenCV

OpenCV là một thư viện học máy và thị giác máy tính mã nguồn mở. Nó cho phép xử lý ảnh và video, phát hiện đối tượng, khuân mặt và chữ viết tay.

Những kiến thức về **OpenCV** tìm hiểu được sẽ được em trình bày trong một file **jupyter** để thuận tiện cho việc tìm hiểu và thực hành

Source: license-plate-ocr/SourceCode/openCV.ipynb at main · hoangvudinh0301/license-plate-ocr

III. Image Preprocessing

Image Preprocessing (Tiền xử lý ảnh) Tiền xử lý ảnh là quá trình cải thiện hình ảnh trước khi thực hiện các bước phân tích hoặc nhận diện. Mục tiêu của tiền xử lý là loại bỏ nhiễu, cải thiện độ tương phản, và chuẩn hóa hình ảnh để tăng cường chất lượng và độ chính xác của các kết quả phân tích sau này.

1. Các biến đổi hình học

Đây là tập hợp các phép biến đổi hình ảnh từ một hình dạng này sang mộ hình dạng khác thông qua việc làm thay đổi phương, chiều, góc, cạnh mà không làm thay đổi nội dung của ảnh.

Định nghĩa:

Một phép biến đổi ảnh hình học sẽ được xác định bởi một ma trận dịch chuyển M. Khi đó bất kì 1 điểm có tọa độ (x, y) trên ảnh gốc thông qua phép biến đổi T sẽ có tọa độ trong khôn gian mới sau dịch chuyển là $T(x, y)$ theo công thức:

$$T(x, y) = M\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_{11}x + a_{12}y \\ a_{21}x + a_{22}y \end{bmatrix}$$

1.1. Phóng đại ảnh (Scale ảnh)

Scale ảnh là việc chúng ra thay đổi kích thước dài, rộng của hình ảnh mà không làm thay đổi tính chất song song của các đoạn thẳng trên ảnh gốc so với trục tọa độ X và Y.

Theo định nghĩa về phép biến đổi hình học thì một biến đổi phóng đại các chiều (x, y) theo hệ số (a_1, a_2). Lúc này ta có ma trận dịch chuyển M là ma trận đường chéo có đường chéo chính là $[a_1, a_2]$. Khi đó phép dịch chuyển sẽ là:

$$T(x, y) = M\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_1 & 0 \\ 0 & a_2 \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_1x \\ a_2y \end{bmatrix}$$

Trong opencv, kích thước của ảnh có để được thay đổi bằng các sử dụng hàm cv2.resize(). Resize thường xuyên được sử dụng trong các mô hình deep learning phân loại ảnh vì mỗi một mô hình đều có một kích thước đầu vào tiêu chuẩn. Chẳng hạn như Lenet là lích thước 32x32x3 và Alexnet là 224x224x3.

1.2. Dịch chuyển ảnh

Dịch chuyển ảnh được thực hiện trong trường hợp mong muốn chuyển ảnh đến các vị trí khác nhau. Phép dịch chuyển ảnh sẽ giữ nguyên tính chất song song của các đoạn thẳng sau khi dịch chuyển đối với các trục X hoặc Y nếu chiếu dịch chuyển cũng song song với một trong hay trục. Khi đó (t_x, t_y) là các giá trị di chuyển ảnh theo hai trục. Khi đó ma trận dịch chuyển M sẽ có dạng

$$M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$$

Khi đó phép biến đổi (x, y) bất kì theo ma trận dịch chuyển M với chiều thứ 3 là một hằng số, chẳng hạn $z = 1$ sẽ là:

$$T(x, y) = M\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \end{bmatrix}$$

Trong opencv, áp dụng hàm cv2.warpAffine() với đầu vào là ma trận dịch chuyển M và ảnh gốc.

1.3. Xoay ảnh (Rotation)

Xoay ảnh là phép quay một bức ảnh theo một góc xác định quanh một điểm. Phép xoay ảnh sẽ không đảm bảo tính chất song song với các trục tọa độ nhưng nó sẽ bảo toàn độ lớn góc. Phép xoay của một ảnh tương ứng với một góc θ đạt được bằng một ma trận dịch chuyển M như sau:

$$M = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

Trong opencv để xoay ảnh chúng ra sẽ sử dụng hàm cv2.getRotationMatrix2D() để tìm ma trận M và áp dụng hàm cv2.wrapAffine() để xoay ảnh.

1.4. Biến đổi Affine

Trong biến đổi affine, toàn bộ các đường thẳng song song trong bức ảnh gốc giữ nguyên tính chất song song ở ảnh đầu ra. Để tìm ma trận chuyển vị, chúng ta cần xác định ra 3 điểm từ ảnh đầu vào và tọa độ tương ứng của chúng trong hình ảnh đầu ra. Hàm cv2.getAffineTransform sẽ tạo ra được một ma trận 2x3 được truyền vào hàm cv2.warpAffine.

1.5. Biến đổi phối cảnh

Để biến đổi phối cảnh thì chúng ta cần một ma trận biến đổi 3x3. Đường thẳng sẽ giữ nguyên là đường thẳng sau biến đổi. Để tìm ra ma trận biến đổi này, chúng ta cần tìm ra 4 điểm trong ảnh đầu vào tương ứng với các điểm trong ảnh đầu ra. Trong số 4 điểm này, không có bất kì 3 điểm nào thẳng hàng. Sau đó ma trận biến đổi có thể được thiết lập thông qua hàm số cv2.getPerspectiveTransform. Và áp dụng cv2.warpPerspective với ma trận biến đổi 3x3.

2. Làm mịn ảnh

2.1. Khái niệm

Làm mịn ảnh (Image Smoothing) là kỹ thuật trong ImageProcessing dùng để giảm nhiễu (noise) và làm ảnh trông mượt hơn bằng các làm thay đổi đột ngội giữa các pixel. Mục tiêu của phương pháp này là cải thiện chất lượng ảnh và tạo điều kiện thuận lời cho các bước xử lý tiếp theo.

Nguyên lý cơ bản của làm mịn ảnh là thay thế giá trị của một pixel bằng giá trị được tính thoán từ các pixel lân cận của nó trong một cửa sổ nhỏ gọi là kernel hoặc filter mask.

2.2. Nguyên lý hoạt động

Trong quá trình làm mịn ảnh, một cửa sổ kích thước $\boldsymbol{m} \times \boldsymbol{n}$ sẽ được di chuyển lần lượt qua toàn bộ ảnh. Tại mỗi vị trí, giá trị pixel trung tâm sẽ được tính lại dựa trên các pixel nằm trong vùng cửa sổ đó. Phép toàn này thường được thực hiện thông qua tích chập (**convolution**) giữa ảnh và một ma trận **kernel**.

Ví dụ với **kernel** $\mathbf{3} \times \mathbf{3}$:

$$\begin{vmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{vmatrix}$$

Giá trị pixel mới được tính bằng trung bình của các pixel trong vùng lân cận.

Các phương pháp làm mịn ảnh phổ biến

2.3. Các phương pháp làm mịn phổ biến

2.3.1. Mean Filter

Mean filter là phương pháp đơn giản nhất trong các kỹ thuật làm mịn ảnh. Phương pháp này thay thế giá trị của pixel bằng giá trị trung bình của các pixel trong vùng lân cận.

Ưu điểm của phương pháp này là dễ thwucj hiện và tốc độ xử lý nhanh. Tuy nhiên, nhược điểm là có thể làm mờ các chi tiết quan trọng trong ảnh, đặt biệt là các cạnh của đối tượng.

2.3.2. Gaussian Filter

Gaussian Filter là phương pháp làm mịn phổ biến trong xử lý ảnh. Phương pháp này sử dụng hàm **Gaussian** để gán trọng số cho các pixel trong kernel, trong đó các pixel gần trung tâm có trọng số lớn hơn so với các pixel ở xa.

Ưu điểm của Gaussian Filter là giúp giảm nhiễu hiệu quả mà vẫn giữ được nhiều chi tiết ảnh. Do đó, phương pháp này thường được sử dụng trước các bước phát hiện cạnh.

2.3.3. Median Filter

Median Filter là phương pháp lọc phi tuyến tính sử dụng để loại bỏ nhiễu, đặc biệt là **nhiễu muối tiêu (salt-and-pepper noise)**. Thay vì tính trung bình, phương pháp này sắp xếp các giá trị pixel trong vùng lân cận theo thứ tự và chọn giá trị trung vị để thay thế cho pixel trung tâm.

Ưu điểm của Median Filter là có khả năng loại bỏ nhiễu tốt mà vẫn giữ được các cạnh của đối tượng trong ảnh.

2.3.4. Bilateral Filter

Bilateral Filter là phương pháp lọc nâng cao có khả năng làm mịn ảnh đồng thời bảo toàn các cạnh. Phương pháp này kết hợp hai yếu tố: khoảng cách không gian giữa các pixel và sự khác biệt về cường độ sáng.

Do khả năng giữ cạnh tốt, Bilaternal Filter thường được sử dụng trong các ứng dụng xử lý ảnh nâng cao và thị giác máy tính.

2.4. Vai trò của làm mịn ảnh trong xử lý ảnh

Làm mịn ảnh đóng vai trò quan trọng trong giai đoạn tiền xử lý ảnh của nhiều hệ thống thị giác máy tính. Việc giảm nhiễu giúp tăng độ chính xác của thuật toán phát hiện đặc trưng và nhận dạng đối tượng.

Trong các bài toán thực tế như nhận diện biển số xe, làm mịn ảnh thường được thực hiện trước các bước như:

- Phát hiện cạnh
- Phân đoạn ảnh
- Nhận dạng ký tự

Nhờ đó, hệ thống có thể hoạt động ổn định và chính xác hơn.

3. Cải thiện độ tương phản